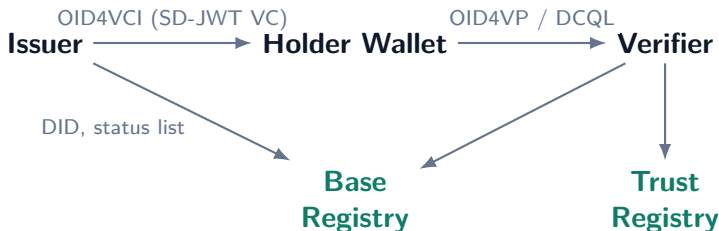

Implementing ZK in swiyu

Antonio Kambiré | zkSecurity | July 2026

How swiyu works

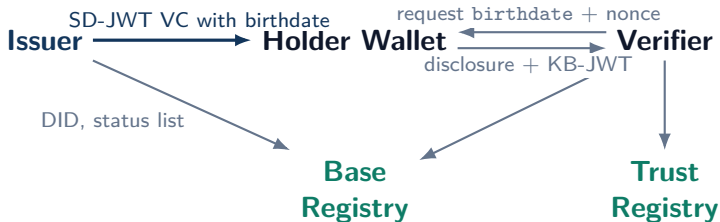
- Issuers are government agencies, companies, and institutions; holders use the swiyu wallet app; verifiers run swiyu Check (in person) or the Generic Verifier (online).
- The Base Registry publishes DIDs (`did:webvh`), public keys, and Token Status Lists; the Trust Registry says which real-world entity a DID belongs to.
- All beta components are open source; the wallet and verifier are the natural extension points.



swiyu infrastructure flow: proving you are over 18

1. Issuance: inside the SD-JWT VC

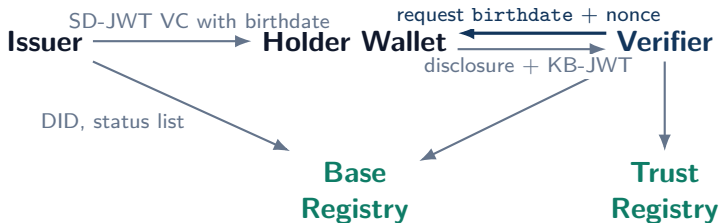
- The signed payload holds iss, vct, exp, status, cnf (wallet key), and claim digests `_sd`: $[H(d_1), H(d_2), \dots]$.
- Each disclosure `[salt, "birthdate", "2001-05-03"]` stays with the wallet and hashes to a digest in `_sd`.
- The signature is ES256 (ECDSA P-256) under the issuer's published DID key.



swiyu infrastructure flow: proving you are over 18

2. Request: OID4VP / DCQL

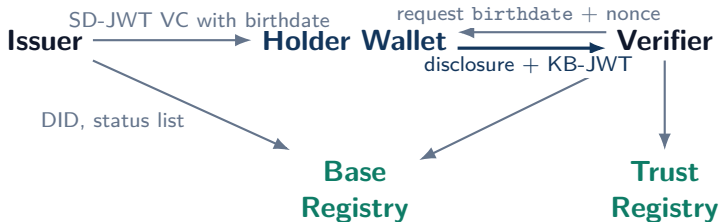
- The verifier names a credential type and the claims it wants (here the birthdate of a Beta-ID) in a DCQL query.
- The request carries a fresh nonce and the verifier identity (audience).
- DCQL selects *which claims to reveal*; it has no way to ask only “over 18?”.



swiyu infrastructure flow: proving you are over 18

3. Presentation: selected disclosures + KB-JWT

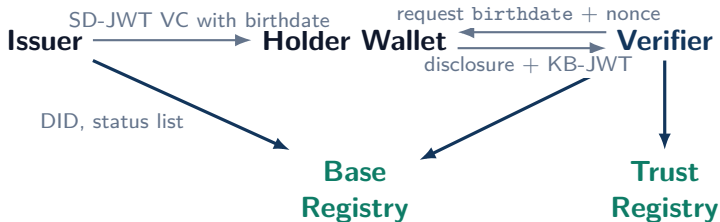
- The wallet returns the issuer-signed JWT and only the selected disclosures; the birthdate is revealed exactly.
- The KB-JWT is a signature with the cnf wallet key over the credential hash, nonce, and audience: it proves holder binding and prevents replay.



swiyu infrastructure flow: proving you are over 18

4. Verification: registries and status

- Resolve the issuer DID in the Base Registry to get its public key; check the Trust Registry statement for who that DID is.
- Verify the ES256 signature, disclosure hashes, and KB-JWT.
- Fetch the Token Status List at the credential's status URI and check its index.



Gap 1: no predicate proofs

What happens today

- A bar only needs “over 18”, but SD-JWT can either withhold the birthdate or reveal it exactly.
- So every age check hands the verifier a full, exact birthdate.

Why it matters

- Verifiers can store what they receive indefinitely.
- Colluding verifiers can join their logs on revealed values; an exact birthdate is a strong join key. swiyu’s threat model names verifier-verifier and issuer-verifier collusion explicitly.

Hiding a claim is not the same as proving something about it.

Gap 2: presentations stay linkable

Stable identifiers in every presentation

- Issuer signature bytes and salted claim digests repeat verbatim each time.
- The cnf holder public key is a stable identifier.
- A status-list URI + bit index unique to the credential lets verifiers re-check it later or cross-match it.
- Any of these seen twice links presentations, with nothing disclosed.

Current remediation: batch issuance

- Issue many one-time credentials for the same attributes, each with a fresh holder key and status index; spend each once.
- Limits: reuse once the batch runs out, the issuer sees renewal timing, each status entry is still re-checkable, and Gap 1 remains.

Batch issuance reduces reuse linkage; it does not deliver full unlinkability.

Why ZK

Target primitive

swiyu SD-JWT VC → challenge-bound ZK proof that the credential was signed by a verifier-accepted issuer key/DID, is within its validity period, is held by a wallet controlling the cnf key, and satisfies a selected predicate: “over 18”, not a birthdate.

This closes both gaps

- Gap 1: the verifier learns the predicate result, never the raw value.
- Gap 2: signature bytes, digests, and holder key stay inside the private witness; each proof is fresh.

And swiyu stays intact

- New work sits in the open components: wallet-side proving and verifier-side validation.
- Issuance, registries, and trust semantics are untouched; ordinary presentations keep working alongside.

EUDI reached the same conclusion: ARF issue #200 recommends anonymous credentials; TS13 explores circuit proofs over existing SD-JWT VCs.

OpenAC (zkID) is the strongest starting point

Why it fits

- SD-JWT-P256 profile matches swiyu exactly: ES256 SD-JWT, disclosures, `cnf.jwk`.
- Prepare/Show: prove the credential once, then per-presentation challenge-bound predicate proofs.
- The other natural options, Crescent (generic JWT/mDL) and Longfellow (mdoc), fit less well.
- Mobile-practical: ~ 3 s one-time Prepare, ~ 0.1 s per Show (iPhone 17); Flock could cut hash costs further.

But not a direct drop-in

- Align profile and claim normalization with `dc+sd-jwt` and the Beta-ID schema.
- Map issuer keys through swiyu DID resolution and trust statements.
- Define OID4VP/DCQL proof carriage; bind challenge and audience.
- ZK adds a new need: nullifiers for proof of unicity, so one credential cannot sign up twice.
- No public third-party audit yet.

Prototype A: SD-JWT-to-ZK overlay

Wallet Prepare

- Parse the swiyu SD-JWT VC
- Verify ES256 signature and disclosures
- Normalize claims; cache the artifact

Wallet Show

- Produce the challenge-bound device signature
- Prove the predicate
- Prove holder-key possession

Verifier

- Validate proof and circuit/profile id
- Check issuer DID/key and trust policy
- Enforce challenge, audience, replay policy

Success

The verifier learns no raw birth date and no stable holder key for an age proof, receives no SD-JWT disclosure bundle, and cannot replay the proof; ordinary swiyu issuance remains unchanged.

Prototype B: status privacy extension

Compare three status designs for the Token Status List gap:

- **Sparse Merkle Tree**: publish a valid/revoked root; the wallet proves inclusion or non-membership locally. Measure proof size, update cost, witness generation.
- **zkID non-membership roots**: evaluate whether the wallet can avoid any credential-specific remote lookup during presentation.
- **EUDI TS13 adjacent-pair revocation**: hidden revocation handle bound into the credential; wallet proves its identifier lies between a signed revoked pair. Requires a status/issuance change.

Success

No stable status URI/index for the verifier, no presentation timing for the issuer or registry, practical mobile witness generation, and verifier checks simple enough for Generic Verifier integration.